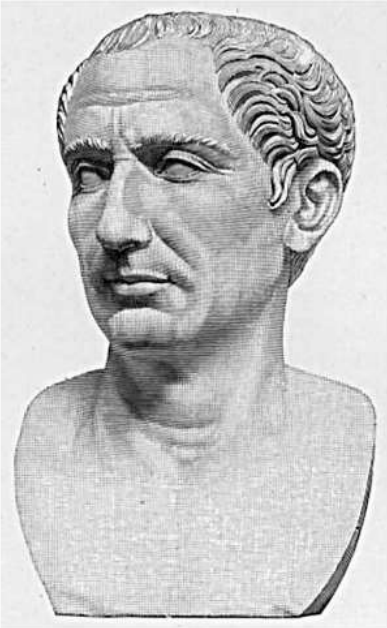
UNIVERSIDAD ADVENTISTA DEL PLATA
FACULTAD DE CIENCIAS ECONÓMICAS Y DE LA ADMINISTRACIÓN
ALGORITMOS Y ESTRUCTURA DE DATOS
TRABAJO PRÁCTICO Nº 2
El Cifrado de César





El Cifrado de César

1. Objetivo del proyecto

Este proyecto tiene como finalidad evaluar la capacidad del estudiante para diseñar y construir un algoritmo completo en Java, integrando los contenidos vistos hasta el momento en la cátedra.

A diferencia del proyecto anterior, en este trabajo la elección de la estructura de datos para almacenar y procesar la información queda a criterio del estudiante: puede optar por arrays tradicionales o por colecciones (ArrayList, HashMap, etc.). Esa decisión deberá ser justificada por escrito, como parte de la entrega.

El proyecto no consiste en solo lograr que el cifrado funcione, sino demostrar criterio técnico en dos aspectos: la organización del código en funciones dentro de una sola clase, y la elección justificada de la estructura de datos. Se valorará tanto la correctitud del algoritmo como la claridad del razonamiento presentado en el texto de justificación.

2. Descripción general

El estudiante deberá desarrollar, en Java y por consola, un programa que implemente el algoritmo de Cifrado César: una técnica clásica de criptografía que consiste en desplazar cada letra de un texto un número fijo de posiciones dentro del alfabeto. El programa debe permitir cifrar y descifrar mensajes ingresados por el usuario, aplicando un desplazamiento (clave) definido también por el usuario.

Por ejemplo, con un desplazamiento de 3, la letra "A" se convierte en "D", la "B" en "E", y así sucesivamente. Al llegar al final del alfabeto, el desplazamiento debe "dar la vuelta" (la "Z" con desplazamiento 3 se convierte en "C").

3. Requisitos técnicos obligatorios

- Lenguaje: Java (consola / entrada por teclado con Scanner).
- Todo el proyecto debe implementarse en una única clase (una sola clase pública con su método main y todos los métodos auxiliares dentro de esa misma clase).
- El programa debe estar organizado en funciones (métodos), evitando repetir lógica: por ejemplo, un método para cifrar, uno para descifrar y, si se desea, métodos auxiliares de validación.
- Estructura de datos: libre elección del estudiante (array tradicional o alguna colección del paquete java.util, como ArrayList). La elección debe ser consistente en todo el programa y debe justificarse en el texto de entrega (ver sección 5).
- El cifrado y el descifrado deben conservar mayúsculas y minúsculas, y no deben modificar espacios, números ni signos de puntuación.

4. Funcionalidades requeridas

4.1 Menú principal

El programa debe mostrar un menú que se repita mediante un bucle hasta que el usuario elija la opción "Salir":

- 1. Cifrar un mensaje
- 2. Descifrar un mensaje
- 3. Descifrar un mensaje sin clave (Fuerza Bruta)
- 4. Salir

4.2 Cifrar un mensaje

1. Solicitar al usuario el texto a cifrar (puede incluir espacios, números y signos de puntuación).
2. Solicitar el desplazamiento (clave), validando que sea un número entero. Se debe aceptar cualquier valor entero (positivo, negativo o mayor a 25) y normalizarlo internamente al rango 0-25.

   Ejemplos (Clave=Desplazamiento): 1001=13, -1=25, -3=23, 120=16
3. Aplicar el desplazamiento letra por letra, conservando mayúsculas/minúsculas y dejando intacto los caracteres que no sean letras.
4. Mostrar el mensaje cifrado resultante.
5. Validación de entrada robusta: el programa no debe romperse (ni lanzar excepciones no controladas) si el usuario ingresa una clave no numérica o un texto vacío.

4.3 Descifrar con clave

- Descifrado directo: el usuario ingresa el mensaje cifrado y la clave utilizada originalmente; el programa aplica el desplazamiento inverso.
- Validación de entrada robusta: el programa no debe romperse (ni lanzar excepciones no controladas) si el usuario ingresa una clave no numérica o un texto vacío.

4.4 Descifrar sin clave

- Descifrado por fuerza bruta: el usuario ingresa el mensaje cifrado sin conocer la clave, y el programa prueba automáticamente los 25 desplazamientos posibles (1 a 25), mostrando cada resultado numerado para que el usuario identifique visualmente cuál tiene sentido.
- Validación de entrada robusta: el programa no debe romperse (ni lanzar excepciones no controladas) si el usuario ingresa un texto vacío.

5. Formato de entrega

- Un único archivo .java con el código fuente, comentado, conteniendo una sola clase.
- Nombre de archivo: Nombre_ Apellido _CifradoCesar.java
- Texto breve donde el estudiante explique y justifique qué estructura de datos eligió (array o colección) para representar el alfabeto, indicando ventajas de esa elección frente a la alternativa descartada.
- Fecha de entrega: 11/09/2026.
- Trabajo individual.

6. Rúbrica de evaluación

Criterio	Puntos	Descripción
Cifrado correcto	20	El desplazamiento se aplica correctamente, conservando mayúsculas/minúsculas y sin alterar caracteres no alfabéticos.
Descifrado directo	15	El desplazamiento inverso reconstruye correctamente el mensaje original.
Descifrado fuerza Bruta	15	El programa muestra correctamente los 25 desplazamientos posibles
Manejo de la clave	10	Acepta claves negativas o mayores a 25 sin romper el cifrado.
Menú y control de flujo	10	Uso correcto de bucles para repetir el menú hasta "Salir".
Organización en funciones (métodos)	10	Lógica dividida en métodos con responsabilidades claras, dentro de una sola clase.
Justificación de la estructura de datos elegida	15	Texto escrito que argumenta la elección (array vs. colección) con criterio técnico.
Legibilidad y buenas prácticas	5	Nombres claros, comentarios, indentación.
Total	100	